# Rainfall Prediction Using Machine Learning and Historical Weather Data: A Regression and Time Series Approach

Mohammed Aathil S (241801164)
Department of Artificial Intelligence and Data Science
Rajalakshmi Engineering College Chennai, India

Mohammed Fayyad J (241801165)
Department of Artificial Intelligence and Data Science
Rajalakshmi Engineering College Chennai, India

Monish Raj RG (241801171)
Department of Artificial Intelligence and Data Science
Rajalakshmi Engineering College Chennai, India

Abstract—Rainfall is irregular, and decisions in agriculture, water management, and flood preparedness are often made on guesswork instead of on what past weather records show. This paper presents the design of a data-driven rainfall estimation study built on historical meteorological observations from a public source such as the India Meteorological Department (IMD) or the National Oceanic and Atmospheric Administration (NOAA). The study will estimate rainfall in millimetres from date, temperature, humidity, pressure, wind speed, and previous rainfall, and will switch to a rain/no-rain classification if the chosen dataset requires it. Three candidate models are compared: a Linear Regression baseline, a Random Forest Regressor, and a seasonal-naive or moving-average time-series baseline, with ARIMA as an optional extension. To avoid data leakage, the data are split chronologically without shuffling, and all preprocessing parameters are learned from the training period only. Models are compared on the same test period using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and the $R^2$ score. An optional large language model layer served through the OpenRouter API will explain predictions in plain language. The project is at the proposal stage: the dataset has not yet been downloaded and no model has been trained, so this paper reports the design and the evaluation protocol, not experimental results.

Index Terms—historical weather data, machine learning, random forest, rainfall prediction, regression, time series analysis.

## I. Introduction

Rainfall is irregular in both timing and amount, and planning that rests on guesswork is risky. Farmers decide when to irrigate, water authorities decide how much to store or release, and flood-preparedness teams decide when to act. Each of them is better served by a data-based estimate than by intuition. Records of past weather already hold patterns worth analysing, and this project sets out to use them.

Two questions are closely related. Will it rain? is a classification problem, and how much rain? is a regression problem. We treat rainfall amount in millimetres as the primary target and estimate

it from historical meteorological variables with regression and time-series methods. If the chosen dataset turns out to suit a rain/no-rain label better, the task will switch to classification and the evaluation will use classification metrics instead. The dataset therefore determines the final output task.

Statistical time-series models such as the autoregressive integrated moving average (ARIMA) family [1] and machine-learning regressors such as the random forest [2] approach the problem differently. Which works better on a given record is an empirical question, and it can be answered fairly only if every model is trained on the same period and scored on the same held-out period. Because weather records are ordered in time, a random train–test split would let a model see the future during training. A chronological split is therefore a central design decision [3], [4].

This paper is a proposal. At the time of writing the dataset has not been downloaded, no model has been trained, and no result has been obtained. We therefore describe the problem, the data plan, the candidate models, the leakage-aware evaluation protocol, and the technology stack, and we state what the study is expected to deliver rather than what it has achieved. Our planned contributions are (i) a defined rainfall-estimation task and data-preparation plan built on a credible public dataset, (ii) a chronological, leakage-aware split and validation scheme, (iii) a like-for-like comparison of a linear baseline, a random forest, and a time-series baseline, with ARIMA as an optional extension, and (iv) an optional language-model layer that explains predictions in plain language.

Section II reviews related work, Section III presents the system design, Section IV explains the methodology, Section V describes the implementation plan, Section VI sets out the evaluation protocol and expected outcomes, Section VII lists limitations and future work, and Section VIII concludes.

## II. Related Work

### A. Time-Series Forecasting

The Box–Jenkins approach models a series through autoregressive and moving-average terms, with differencing to remove trend [1]. Simple benchmarks, namely the seasonal naive forecast and the moving average, are standard reference points in forecasting practice, because a more elaborate model is worth using only if it beats them [5]. Prophet [6] fits an additive model with trend and seasonal components and is a possible optional addition to this study.

### B. Regression and Ensemble Methods

Linear regression is the classical interpretable model for a continuous target [7]. A random forest averages many decorrelated decision trees grown on bootstrap samples and can capture nonlinear relationships and interactions among inputs without a hand-specified functional form [2]. Both are available in scikit-learn [8], which the project will use for modelling, scaling, and metrics.

### C. Evaluating Models on Ordered Data

Evaluation on time-ordered data needs more care than evaluation on independent samples. Kaufman et al. [3] formalise data leakage, in which information that would be unavailable at prediction time reaches the training process. Bergmeir and Benítez [4] study how cross-validation should be adapted for time-series

predictors, and Cerqueira et al. [9] compare performance-estimation methods for forecasting empirically. These works motivate our chronological split and forward-chaining validation. On the choice of error measure, Willmott and Matsuura [10] discuss the relative merits of MAE and RMSE as measures of average error, which is why we report both.

### D. Historical Weather Data

National meteorological services publish historical observations. The India Meteorological Department [11] and the NOAA National Centers for Environmental Information [12] are two such sources. Either one, or an equivalent public dataset, is a candidate for this study. Our emphasis is on a transparent, reproducible pipeline and a fair model comparison under a leakage-aware protocol.

## III. System Design

### A. Requirements

We set the following requirements before implementation. The study must use a credible historical weather dataset. The time order of the data must be preserved, and preprocessing must be learned from training data only. Every model must be compared on the same test period, and every feature must be available at prediction time. The pipeline must run in a Jupyter notebook on ordinary hardware, so no graphics processing unit (GPU) is assumed. Anything beyond this core, such as a dashboard or language-model explanations, is optional.

### B. Workflow

The workflow has ten stages (Fig. 1), read down the left column and then down the right. Historical data are cleaned, explored, and turned into features. The data are then split chronologically, the models are trained and evaluated on the same test period, and rainfall predictions are produced. An optional stage passes the predictions to a language model for a plain-language explanation, and the final stage visualises and interprets the results.

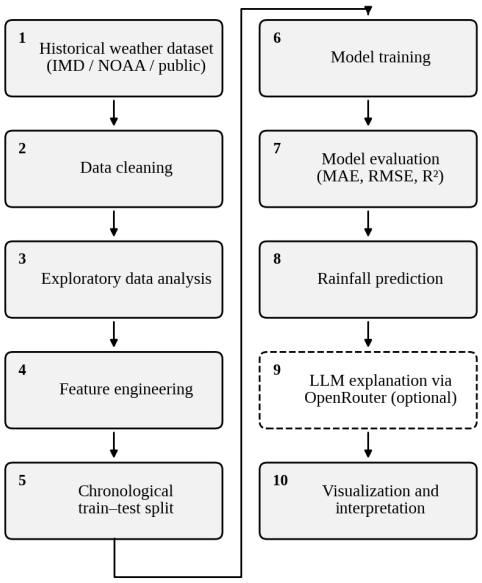



Fig. 1. Planned workflow of the rainfall prediction study. The dashed box marks an optional stage.

### C. Technology Stack

Table I lists the planned tools. Python is the core language and runs in Jupyter [13]. Pandas [14] and NumPy [15] handle loading, cleaning, and numeric work, and Matplotlib [16] and Seaborn [17] produce exploratory and result plots. Scikit-learn [8] provides regression, scaling, and metrics, and Statsmodels [18] provides time-series models. Git and GitHub are optional for version control.

TABLE I
PLANNED TECHNOLOGY STACK

| Layer | Technology | Role |
|---|---|---|
| Language | Python in Jupyter | Core language, notebooks |

| Layer | Technology | Role |
|---|---|---|
| Data preparation | pandas, NumPy | Loading, cleaning, numerics |
| Exploration | Matplotlib, Seaborn | EDA and result plots |
| Modeling | scikit-learn, Statsmodels; Prophet (optional) | Regression, scaling, metrics; time series |
| Data source | IMD / NOAA / public dataset | Historical weather records |
| Explanation | OpenRouter API (optional) | LLM summaries of predictions |
| Tooling | Git, GitHub (optional) | Version control |

### D. Optional Explanation Layer

An optional layer calls a large language model (LLM) through the OpenRouter API [19] to explain predictions and summarize results in plain language. It sits after the prediction stage and receives numbers the models have already produced, so it cannot change predicted values or the reported metrics. Its text will be checked against those numbers, because a generated explanation can be fluent and still wrong.

## IV. Methodology

### A. Data Acquisition

A historical weather dataset from a credible source such as IMD or NOAA, or a public equivalent, will be selected. We will prefer a dataset that covers several seasonal cycles, contains a rainfall field in millimetres, and has few missing values. Dataset selection also decides the output task: a rainfall-in-millimetres field supports regression, whereas a rain/no-rain field supports classification. Table II lists the candidate features and targets. Calendar variables may be derived from the date during feature engineering.

TABLE II
Candidate Features and Targets

| Variable | Role | Note |
|---|---|---|
| Date | Time index | Preserves chronological order |
| Temperature | Feature | Meteorological input |
| Humidity | Feature | Meteorological input |
| Pressure | Feature | Meteorological input |
| Wind speed | Feature | Meteorological input |
| Previous rainfall | Feature | Earlier observations only |
| Rainfall (mm) | Target | Used for regression |
| Rain yes/no | Target | Used if classification is chosen |

### B. Preprocessing

Preprocessing has three parts. First, records are cleaned: missing values are handled, duplicates are removed, data types are fixed, and outliers are detected. Second, features are prepared: categorical variables are encoded and features are scaled where a model needs it. Every transformation is fitted on the training data only and then applied unchanged to the test data. Third, the time order is preserved throughout. Exploratory data analysis (EDA) precedes

modelling to reveal patterns such as seasonality. Rainfall series often contain many dry observations with zero rainfall and a long tail of heavy events, so we will check this property in the EDA and let it inform the interpretation of the metrics.

## C. Chronological Split

The data are split by time, with earlier records used for training and the latest records held out for testing, and no shuffling (Fig. 2a). Shuffling would put future observations in the training set and make the test error look better than it would be in use. Where a model has hyperparameters, they will be tuned by forward-chaining validation inside the training period (Fig. 2b), in which each validation block lies after its training block [4]. The split proportions will be fixed once the length of the chosen record is known.

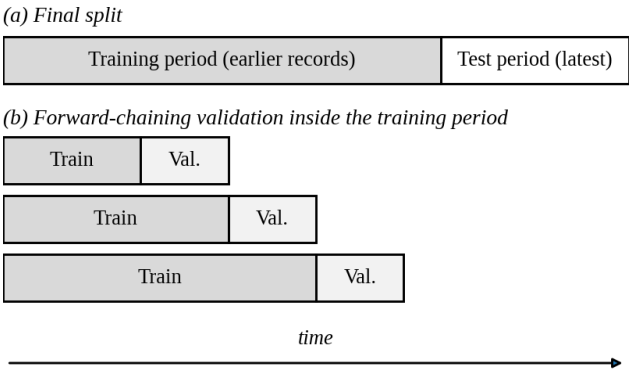



Fig. 2. Chronological train–test split (a) and forward-chaining validation within the training period (b).

## D. Candidate Models

Three candidate models are compared (Table III). These are candidates only: no model has been trained or selected yet.

TABLE III
CANDIDATE MODELS

| Model | Role | Captures |
|---|---|---|
| Linear Regression | Interpretable baseline | Linear effects of weather features |
| Random Forest Regressor | Main ML model | Nonlinear relationships and interactions |
| Seasonal naive / moving average | Time-series baseline | Seasonality and recent level |
| ARIMA (optional) | Optional extension | Autocorrelation and trend |

Linear Regression is an interpretable baseline for rainfall amount from weather features. For p features it predicts

$$\hat{y} = \beta_0 + \sum_{j=1}^{p} \beta_j x_j \tag{1}$$

with coefficients fitted by least squares [7]. The Random Forest Regressor captures nonlinear relationships among weather features. With B trees $T_b$ it predicts

$$\hat{y}(x) = \frac{1}{B} \sum_{b=1}^{B} T_b(x) \tag{2}$$

The time-series baseline uses temporal patterns and seasonality. A seasonal naive forecast repeats the value observed one season earlier, and a moving-average forecast averages the last k observations:

$$\hat{y}_t = y_{t-m} \tag{3}$$

$$\hat{y}_t = \frac{1}{k} \sum_{i=1}^{k} y_{t-i} \tag{4}$$

where m is the seasonal period. ARIMA [1] is optional if scope allows; it combines autoregressive terms of order p, differencing of order d, and moving-average terms of order q.

### E. Evaluation Metrics

Regression models are scored with Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and the $R^2$ score where appropriate. For N test observations with observed rainfall $y_i$, prediction $\hat{y}_i$, and mean $\bar{y}$:

$$\mathrm{MAE} = \frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i| \tag{5}$$

$$\mathrm{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2} \tag{6}$$

$$R^2 = 1 - \frac{\sum_{i=1}^{N}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{N}(y_i - \bar{y})^2} \tag{7}$$

Lower MAE and RMSE generally mean smaller errors. RMSE penalises large errors more heavily than MAE, so a wide gap between them signals occasional large misses, such as heavy-rain events [10]. If classification is selected, classification metrics such as accuracy, precision, recall, and F1 score will replace these.

## V. Implementation Plan

The study will be carried out in Python in Jupyter notebooks, following the stages of Fig. 1: obtain the dataset, clean it, explore it, engineer features, split it chronologically, train the candidate models, evaluate them on the common test period, and produce predictions and plots. Git and GitHub may be used for version control. The optional OpenRouter explanation step and the optional Streamlit dashboard [20] will be added only if time allows.

## VI. Evaluation Protocol and Expected Outcomes

### A. Protocol

All candidate models will be trained on the same training period and scored on the same test period, so that differences in error come from the models and not from the data. We will report MAE, RMSE, and $R^2$ for every model in one table and plot predicted against observed rainfall over the test period. The time-series baseline serves as a sanity check: a model that cannot beat a seasonal naive or moving-average forecast has learned little beyond the seasonal pattern.

### B. Expected Deliverables

The study is expected to deliver an evaluated prediction model, predicted rainfall values, trend visualisations, and a model-performance comparison. These are planned outputs and not achieved results.

### C. Potential Applications

If the models perform well enough, rainfall estimates could support agriculture and irrigation planning, water resource management, and disaster and flood preparedness. Whether the accuracy reached is sufficient for any such use is a question the evaluation must answer first.

### D. Threats to Validity

Four points will limit how far the results can be pushed. First, results from one dataset or location may not carry over to other regions or climates. Second, a single chronological test period may be unusually wet or dry, so the error figures describe that period only. Third, some of any model's skill may come from seasonality alone, which is why the time-series baseline matters. Fourth, gaps in measurement and changes in

recording practice in historical archives may affect data quality.

## VII. Limitations and Future Work

This paper is a design document. The dataset has not been downloaded, no preprocessing has been run, and no model has been trained, so no claim about accuracy can be made. The task itself, regression or classification, may change once the dataset is chosen, and the candidate models and metrics would change with it. The feature list is limited to variables found in typical public archives and may shrink further depending on what the chosen dataset provides.

Future scope includes more weather stations, better seasonal features, and the study of extreme rainfall events. An optional Streamlit dashboard [20] and optional LLM explanations via OpenRouter [19] are also planned extensions. Prophet [6] and ARIMA [1] may be added if scope allows.

## VIII. Conclusion

We have described a planned study that estimates rainfall from historical weather data using regression and time-series methods. Its main design choices are a credible public dataset, a chronological split with preprocessing fitted on training data only, and a like-for-like comparison of Linear Regression, a Random Forest Regressor, and a time-series baseline under MAE, RMSE, and $R^2$. The study has not yet been carried out, so the next steps are to select the dataset, run the pipeline, and replace the expected outcomes in this paper with measured results. Data science can reveal patterns in historical weather data and support rainfall estimation; whether these models do so well enough to be useful is what the experiments are meant to show.

## References


[1] G.E P. Box, G. M. Jenkins, G. C. Reinsel, and G.M.Ljung, Time Series Analysis: Forecasting and Control, 5th ed. Hoboken, NJ, USA: Wiley, 2015.

[2] L. Breiman, “Random forests,” Mach. Learn., vol. 45, no. 1, pp. 5–32, Oct. 2001.

[3] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, “Leakage in data mining: Formulation, detection, and avoidance,” ACM Trans. Knowl. Discovery Data, vol. 6, no. 4, art. 15, Dec. 2012.

[4] C. Bergmeir and J. M. Benítez, “On the use of cross-validation for time series predictor evaluation,” Inf. Sci., vol. 191, pp. 192–213, May 2012.

[5] R. J. Hyndman and G. Athanasopoulos, Forecasting: Principles and Practice, 3rd ed. Melbourne, Australia: OTexts, 2021.

[6] S. J. Taylor and B. Letham, “Forecasting at scale,” Amer. Statistician, vol. 72, no. 1, pp. 37–45, 2018.

[7] T. Hastie, R. Tibshirani, and J. Friedman, The Elements of Statistical Learning, 2nd ed. New York, NY, USA: Springer, 2009.

[8] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” J. Mach. Learn. Res., vol. 12, pp. 2825–2830, 2011.

[9] V. Cerqueira, L. Torgo, and I. Mozetič, “Evaluating time series forecasting models: An empirical study on performance estimation methods,” Mach. Learn., vol. 109, no. 11, pp. 1997–2028, Nov. 2020.

[10] C. J. Willmott and K. Matsuura, “Advantages of the mean absolute error

(MAE) over the root mean square error (RMSE) in assessing average model performance," Climate Res., vol. 30, no. 1, pp. 79–82, 2005.

[11] India Meteorological Department, Ministry of Earth Sciences, Government of India. [Online]. Available: https://mausam.imd.gov.in

[12] NOAA National Centers for Environmental Information, "Climate Data Online." [Online]. Available: https://www.ncei.noaa.gov/cdo-web/

[13] T. Kluyver et al., "Jupyter Notebooks—A publishing format for reproducible computational workflows," in Proc. 20th Int. Conf. Electron. Publishing (ELPUB), Göttingen, Germany, 2016, pp. 87–90.

[14] W. McKinney, "Data structures for statistical computing in Python," in Proc. 9th Python Sci. Conf. (SciPy), Austin, TX, USA, 2010, pp. 56–61.

[15] C. R. Harris et al., "Array programming with NumPy," Nature, vol. 585, no. 7825, pp. 357–362, Sep. 2020.

[16] J. D. Hunter, "Matplotlib: A 2D graphics environment," Comput. Sci. Eng., vol. 9, no. 3, pp. 90–95, May/Jun. 2007.

[17] M. L. Waskom, "seaborn: Statistical data visualization," J. Open Source Softw., vol. 6, no. 60, art. 3021, 2021.

[18] S. Seabold and J. Perktold, "Statsmodels: Econometric and statistical modeling with Python," in Proc. 9th Python Sci. Conf. (SciPy), Austin, TX, USA, 2010, pp. 92–96.

[19] OpenRouter, "OpenRouter documentation." [Online]. Available: https://openrouter.ai/docs

[20] Streamlit Inc., "Streamlit documentation." [Online]. Available: https://docs.streamlit.io